



UNIVERSIDADE FEDERAL DO TOCANTINS
CAMPUS UNIVERSITÁRIO DE PALMAS
CURSO DE CIÊNCIA DA COMPUTAÇÃO

Laboratório 2 – Display de 7 Segmentos e Conversor Gray-Binário

Pedro Lucas Moreira

Relatório – Linguagens de Descrição de Hardware – 2026/2 – Setembro de
2026

Professor(a) Orientador(a): Tiago Almeida

The contents of this report are the sole responsibility of the authors.
O conteúdo deste relatório é de única responsabilidade dos autores.

Pedro Lucas Moreira

Laboratório 2 – Display de 7 Segmentos e Conversor Gray-Binário

Relatório apresentado como parte das atividades da disciplina Linguagens de Descrição de Hardware – 2026/2, Curso de Ciência da Computação, Universidade Federal do Tocantins.

PALMAS / TO

2026

1 QUESTÃO 1 – DISPLAY DE 7 SEGMENTOS

1.1 Item (a) – Decodificador decimal com detecção de erro

Cada bit de SWITCH_PINS corresponde a um switch físico da DE1 (0 = chave para baixo, 1 = chave para cima), formando um número binário de 0 a 127. O decodificador mapeia esse valor diretamente para os segmentos do display: as combinações de 0 a 9 acendem o dígito correspondente, e qualquer valor fora dessa faixa (10 a 127) acende o padrão da letra E.

```
1 library ieee;
2 use ieee.std_logic_1164.all;
3
4 entity display is
5     port (
6         SWITCH_PINS: in std_logic_vector(6 downto 0);
7         HEX_PINS: out std_logic_vector(6 downto 0)
8     );
9 end entity display;
10
11 architecture rtl of display is
12 begin
13     with SWITCH_PINS select
14         HEX_PINS <= "1000000" when "0000000", — 0
15                     "1111001" when "0000001", — 1
16                     "0100100" when "0000010", — 2
17                     "0110000" when "0000011", — 3
18                     "0011001" when "0000100", — 4
19                     "0010010" when "0000101", — 5
20                     "0000010" when "0000110", — 6
21                     "1111000" when "0000111", — 7
22                     "0000000" when "0001000", — 8
23                     "0010000" when "0001001", — 9
24                     "0000110" when others; — E (fora da faixa 0–9)
25
26 end architecture rtl;
```

1.2 Item (b) – Conversor hexadecimal

Para o item (b), a faixa de entrada de 4 bits (0 a 15) é totalmente válida em hexadecimal, então não existe caso de erro: cada combinação binária tem um dígito ou letra (0–9, A–F) correspondente nos segmentos.

```

1  library ieee;
2  use ieee.std_logic_1164.all;
3
4  entity conv_7seg is
5      port (
6          HEX_IN  : in  std_logic_vector(3 downto 0);
7          HEX_OUT : out std_logic_vector(6 downto 0)
8      );
9  end entity conv_7seg;
10
11 architecture rtl of conv_7seg is
12 begin
13     with HEX_IN select
14         HEX_OUT <= "1000000" when "0000", — 0
15                    "1111001" when "0001", — 1
16                    "0100100" when "0010", — 2
17                    "0110000" when "0011", — 3
18                    "0011001" when "0100", — 4
19                    "0010010" when "0101", — 5
20                    "0000010" when "0110", — 6
21                    "1111000" when "0111", — 7
22                    "0000000" when "1000", — 8
23                    "0010000" when "1001", — 9
24                    "0001000" when "1010", — A
25                    "0000011" when "1011", — b
26                    "1000110" when "1100", — C
27                    "0100001" when "1101", — d
28                    "0000110" when "1110", — E
29                    "0001110" when "1111", — F
30                    "1111111" when others;
31
32 end architecture rtl;

```

O topo do projeto (displayb.vhd) apenas instancia o componente conv_7seg,

ligando as entradas físicas dos switches à sua porta, para teste na DE1:

```
1  library ieee;  
2  use ieee.std_logic_1164.all;  
3  
4  entity displayb is  
5      port (  
6          SWITCH_PINS: in std_logic_vector(3 downto 0);  
7          HEX_PINS: out std_logic_vector(6 downto 0)  
8      );  
9  end entity displayb;  
10  
11 architecture rtl of displayb is  
12 begin  
13     conv_inst: entity work.conv_7seg  
14         port map (  
15         HEX_IN  => SWITCH_PINS,  
16         HEX_OUT => HEX_PINS  
17     );  
18  
19 end architecture rtl;
```

2 QUESTÃO 2 – CONVERSOR GRAY – BINÁRIO

2.1 Item (a)

Como cada bit de saída depende apenas dos bits de entrada (nunca de outro bit de saída), a implementação é uma atribuição concorrente direta, sem necessidade de sinal intermediário nem de processo.

```

1  library ieee ;
2  use ieee.std_logic_1164.all ;
3
4  entity gray2bin is
5      port (
6          GRAY_IN : in    std_logic_vector(3 downto 0);
7          BIN_OUT  : out  std_logic_vector(3 downto 0)
8      );
9  end entity gray2bin ;
10
11 architecture rtl of gray2bin is
12 begin
13     BIN_OUT(3) <= GRAY_IN(3) ;
14     BIN_OUT(2) <= GRAY_IN(2) xor GRAY_IN(3) ;
15     BIN_OUT(1) <= GRAY_IN(1) xor GRAY_IN(2) xor GRAY_IN(3) ;
16     BIN_OUT(0) <= GRAY_IN(0) xor GRAY_IN(1) xor GRAY_IN(2) xor GRAY_IN
17                     (3) ;
18 end architecture rtl ;

```

2.2 Itens (d) e (e)

O demo_setup foi feito em VHDL, instanciando os dois componentes já prontos gray2bin e conv_7seg e ligando-os em cascata: a entrada de 4 bits em código Gray (SW) é convertida para binário e em seguida decodificada para os 7 segmentos (HEX3).

```

1  library ieee ;
2  use ieee.std_logic_1164.all ;
3
4  entity demo_setup is

```

```

5      port (
6          SW      : in  std_logic_vector(3 downto 0);
7          HEX3     : out std_logic_vector(6 downto 0)
8      );
9  end entity demo_setup;
10
11  architecture rtl of demo_setup is
12      signal bin_value : std_logic_vector(3 downto 0);
13  begin
14      gray_inst: entity work.gray2bin
15          port map (
16              GRAY_IN => SW,
17              BIN_OUT => bin_value
18          );
19
20      seg_inst: entity work.conv_7seg
21          port map (
22              HEX_IN  => bin_value ,
23              HEX_OUT => HEX3
24          );
25
26  end architecture rtl;

```

2.3 Item (f)

A atribuição de pinos foi feita via script Tcl, cobrindo as 4 entradas de SW e os 7 segmentos de HEX3 conforme a tabela oficial de pinos da DE1:

```

1  package require ::quartus::project
2  package require ::quartus::flow
3
4  set project_name "demo_setup"
5
6  project_new $project_name --overwrite
7
8  set_global_assignment -name FAMILY "Cyclone II"
9  set_global_assignment -name DEVICE EP2C20F484C7
10 set_global_assignment -name VHDL_FILE a.vhd
11 set_global_assignment -name VHDL_FILE ../01/conv_7seg.vhd

```

```
12 set_global_assignment -name VHDL_FILE demo_setup.vhd
13 set_global_assignment -name TOP_LEVEL_ENTITY demo_setup
14
15 set_location_assignment PIN_L22 -to SW[0]
16 set_location_assignment PIN_L21 -to SW[1]
17 set_location_assignment PIN_M22 -to SW[2]
18 set_location_assignment PIN_V12 -to SW[3]
19
20 set_location_assignment PIN_F4 -to HEX3[0]
21 set_location_assignment PIN_D5 -to HEX3[1]
22 set_location_assignment PIN_D6 -to HEX3[2]
23 set_location_assignment PIN_J4 -to HEX3[3]
24 set_location_assignment PIN_L8 -to HEX3[4]
25 set_location_assignment PIN_F3 -to HEX3[5]
26 set_location_assignment PIN_D4 -to HEX3[6]
27
28 export_assignments
29 execute_flow -compile
30
31 project_close
```